

TerraWatch

Plataforma Inteligente de Monitorização de Ocorrências em Portugal

Universidade Fernando Pessoa — Projeto Integrado S2-2025/26

IART · PAW · PSR

Autores: oshico & NunoSilva24

Plataforma em produção: <https://terrawatch.oshico.org>

Índice

1. Descrição do Projeto e Tecnologias
2. Arquitetura do Sistema
3. IART — Inteligência Artificial
4. PAW — Projeto de Aplicações Web
5. PSR — Projeto em Sistemas e Redes
6. Resumo das Tecnologias Utilizadas

1. Descrição do Projeto

1.1 Enquadramento

O TerraWatch é uma plataforma web inteligente de monitorização de ocorrências em Portugal, desenvolvida no âmbito do Projeto Integrado S2-2025/26 da Universidade Fernando Pessoa. A plataforma integra engenharia de dados, machine learning, desenvolvimento web moderno e DevOps, estando atualmente disponível em produção em <https://terrawatch.oshico.org>.

1.2 Funcionalidades Implementadas

- Monitorização de incêndios e dados de precipitação em Portugal
- Mapa interativo com marcação geográfica de ocorrências (Leaflet)
- Dashboard analítico com estatísticas por distrito, ano, mês e causa
- Importação de dados via ficheiros XLSX
- Módulo de Machine Learning com previsão temporal por distrito/cidade
- Autenticação segura com JWT (access + refresh tokens)
- Deployment automatizado com CI/CD via GitHub Actions

1.3 Fontes de Dados

- Dados históricos de incêndios em Portugal (importados via XLSX)
- Dados de precipitação por cidade/ano (importados via XLSX)
- Base de dados PostgreSQL + PostGIS para dados geográficos

2. Arquitetura do Sistema

A arquitetura do TerraWatch segue um modelo de microserviços containerizados, composto por quatro serviços principais comunicando através de uma rede Docker interna, com exposição pública gerida pelo reverse proxy Traefik com TLS automático.

2.1 Diagrama de Componentes

Camada	Serviço / Componente	Tecnologia
Dados	Base de dados relacional + geográfica	PostgreSQL 16 + PostGIS
Armazenamento	Object storage para modelos ML	MinIO (S3-compatible)
Backend API	REST API com autenticação JWT	Go 1.25, Gin, GORM
ML Service	Previsão temporal de incêndios e chuva	Python 3.12, FastAPI, Prophet
Frontend	SPA com mapa interativo e dashboard	React 19, TypeScript 6, Leaflet
Proxy / TLS	Roteamento HTTPS com certificados automáticos	Traefik + Cloudflare
CI/CD	Pipeline automático de lint, build, deploy	GitHub Actions
Orquestração	Docker Compose (dev e produção)	Docker

2.2 Fluxo de Dados

- Os dados são importados por utilizadores autenticados via upload XLSX no frontend
- O backend Go valida, processa e persiste os dados na base PostgreSQL/PostGIS
- O motor de ML lê os dados da mesma base, treina modelos Prophet e guarda-os no MinIO
- O frontend consome as APIs REST do backend e do motor de ML para visualização
- O Traefik gere o encaminhamento HTTPS para cada serviço com base no subdomínio

2.3 Modelo de Dados (principal)

- **fires:** year, month, day, district, lat, long, cause_group, cause_description, alert_source, ...
- **rains:** year, city, precipitation_mm, lat, long
- **users:** id (UUID), email, password_hash, name
- **refresh_tokens:** token, user_id, expires_at, revoked
- **predictions:** type, region, lat, long, predicted_date, value, confidence_lower, confidence_upper, model_name, model_version

3. IART — Inteligência Artificial

3.1 Formulação do Problema

O problema central de ML no TerraWatch consiste na **previsão temporal do número de incêndios por distrito** e da **precipitação futura por cidade** em Portugal. Trata-se de um problema de regressão em séries temporais, onde a variável-alvo é uma contagem (incêndios/mês por distrito) ou um valor contínuo (mm de precipitação/ano por cidade).

3.2 Preparação e Engenharia de Features

O preprocessamento é implementado em `predictionEngine/app/data/preprocessor.py`:

- **Incêndios:** Os dados brutos (year, month, day, district) são agrupados por distrito e período mensal. Cada série temporal é convertida no formato Prophet (colunas 'ds' e 'y'), onde 'y' representa a contagem de incêndios no mês.
- **Precipitação:** Os dados (year, city, precipitation_mm) são agrupados por cidade, com 'ds' como 1 de janeiro de cada ano e 'y' como o valor de precipitação em mm.
- Coordenadas geográficas (lat, long) são calculadas como média por distrito/cidade para georreferenciação das previsões.
- Valores nulos e distritos/cidades com menos de 2 registos são descartados (impossível treinar Prophet).

3.3 Modelo: Facebook Prophet

O modelo escolhido é o **Facebook Prophet**, uma biblioteca de previsão de séries temporais desenvolvida para lidar com sazonalidade, feriados e tendências não-lineares:

- **Sazonalidade anual** ativada em ambos os modelos (padrões anuais de incêndios e chuva)
- **Sazonalidade semanal** ativada apenas nos incêndios (variação dentro da semana)
- **Intervalo de confiança** de 80% (interval_width=0.80) — retorna yhat_lower e yhat_upper
- **Granularidade:** mensal para incêndios (freq='MS'), anual para precipitação (freq='YS')

Parametrização do modelo de incêndios:

```
Prophet(yearly_seasonality=True, weekly_seasonality=True, daily_seasonality=False, interval_width=0.80)
```

3.4 Treino e Persistência

O motor de ML (FastAPI em Python) expõe endpoints REST:

Endpoint	Método	Descrição
POST /train/fires	POST	Treina modelos Prophet para todos os distritos com dados
POST /train/rains	POST	Treina modelos Prophet para todas as cidades com dados de chuva
GET /predictions/fires	GET	Retorna previsões mensais de incêndios (parâmetros: district, months)
GET /predictions/rains	GET	Retorna previsões anuais de precipitação (parâmetros: city, years)

Endpoint	Método	Descrição
GET /models	GET	Lista modelos treinados e guardados no MinIO
GET /health	GET	Estado do serviço e número de modelos carregados

Os modelos treinados são serializados e persistidos no **MinIO** (object storage S3-compatible), permitindo versionamento (versão com timestamp) e reutilização sem necessidade de re-treino. O carregamento é feito on-demand: se o modelo para um dado distrito/cidade não estiver em memória, é carregado do MinIO; se não existir, é treinado automaticamente.

3.5 Output e Avaliação

Para cada região prevista, o modelo retorna:

- **predicted_count** (incêndios) ou **predicted_precipitation_mm** (chuva): valor central previsto
- **confidence_lower** e **confidence_upper**: intervalo de confiança de 80%
- **lat** e **long**: coordenadas para visualização no mapa

As previsões são também persistidas na tabela **predictions** da base de dados, com referência ao modelo e versão usados, permitindo auditoria e análise histórica das previsões.

3.6 Tecnologias IART

Tecnologia	Versão	Utilização
Python	3.12+	Linguagem do motor de ML
FastAPI	>=0.115	Framework REST para o motor de ML
Prophet (Facebook)	>=1.1.6	Modelo de previsão de séries temporais
Pandas	>=2.2	Manipulação e pré-processamento de dados
NumPy	>=2.0	Operações numéricas
SQLAlchemy	>=2.0	Acesso à base de dados PostgreSQL
MinIO (Python SDK)	>=7.2	Persistência de modelos treinados
Uvicorn	>=0.32	Servidor ASGI para FastAPI

3.7 Rubricas IART — Autoavaliação

Critério	Peso	Implementação
Formulação do problema	10%	Previsão de séries temporais (incêndios/chuva) claramente definida
Preparação e engenharia de features	20%	Pré-processamento por distrito/cidade, agrupamento mensal/anual, limpeza de nulos
Escolha e implementação de modelos	25%	Prophet com sazonalidade configurada por tipo de dado
Avaliação experimental	25%	Intervalo de confiança 80%, yhat_lower/yhat_upper, persistência de resultados

Critério	Peso	Implementação
Interpretação e discussão dos resultados	20%	Previsões com coordenadas geográficas, visualizadas no mapa e guardadas em BD

4. PAW — Projeto de Aplicações Web

4.1 Arquitetura da Aplicação Web

O TerraWatch segue uma arquitetura cliente-servidor clara: um **backend REST em Go** e uma **Single Page Application (SPA) em React**. A separação de responsabilidades é rigorosa: o backend não conhece o frontend e vice-versa; a comunicação é exclusivamente via API REST com JSON.

4.2 Backend (Go + Gin)

O backend está estruturado seguindo o padrão **cmd / internal** do Go:

- **cmd/main.go**: ponto de entrada, inicialização do servidor Gin
- **internal/routes/**: definição centralizada de todas as rotas
- **internal/handlers/**: lógica de cada endpoint (fire_locations, rain_locations, import_fire, import_rain, auth_handler, fire_status, fire_dropdowns, import_status, import_task)
- **internal/services/**: lógica de negócio (auth_service)
- **internal/models/**: structs Go mapeadas à BD via GORM (fire, rain, user, refresh_token)
- **internal/db/**: inicialização da ligação PostgreSQL
- **internal/middleware/**: middleware de autenticação JWT

4.3 API REST — Endpoints

Método	Endpoint	Auth	Descrição
GET	/api/v1/ping	—	Health check
GET	/api/v1/locations/fires	—	Localizações de incêndios (filtros: startDate, endDate, local, year, month, day)
GET	/api/v1/locations/rains	—	Localizações de dados de precipitação
GET	/api/v1/stats/fires/by-district	—	Estatísticas de incêndios por distrito
GET	/api/v1/stats/fires/by-year	—	Estatísticas de incêndios por ano
GET	/api/v1/stats/fires/by-cause-group	—	Estatísticas por grupo de causa
GET	/api/v1/stats/fires/by-month	—	Estatísticas de incêndios por mês
GET	/api/v1/meta/cause-groups	—	Lista de grupos de causas disponíveis
GET	/api/v1/meta/cause-descriptions	—	Lista de descrições de causas
GET	/api/v1/meta/alert-sources	—	Lista de fontes de alerta
POST	/api/v1/import/fires	JWT	Importar incêndios via XLSX
POST	/api/v1/import/rains	JWT	Importar chuva via XLSX
GET	/api/v1/import/status/:taskID	JWT	Estado de uma tarefa de importação
POST	/api/v1/auth/register	—	Registo de utilizador
POST	/api/v1/auth/login	—	Login, retorna access_token e refresh_token

Método	Endpoint	Auth	Descrição
POST	/api/v1/auth/refresh	—	Renovar access token
POST	/api/v1/auth/logout	—	Invalidar refresh token
GET	/api/v1/auth/me	JWT	Dados do utilizador atual

4.4 Autenticação JWT

O sistema de autenticação implementa o padrão moderno de **access token + refresh token**:

- Access token de curta duração (configurável, padrão 15 min) para autenticação de pedidos
- Refresh token de longa duração (padrão 720h) persistido na base de dados
- Logout invalida o refresh token na BD (revogação explícita)
- Middleware AuthRequired() valida o token Bearer em todos os endpoints protegidos
- Passwords com hash seguro via bcrypt (golang.org/x/crypto)

4.5 Frontend (React + TypeScript)

O frontend é uma SPA construída com React 19 e TypeScript, organizada por features:

- **features/fires/**: tab de incêndios com hook useFiresData, filtros e mapa
- **features/rain/**: tab de precipitação com hook useRainData
- **features/prediction/**: tab de previsões ML com hook usePredictions
- **features/dashboard/**: dashboard analítico com gráficos e importação de dados
- **features/auth/**: contexto de autenticação, modais de login/registo, rotas protegidas
- **features/map/**: componente MapView com Leaflet para visualização geográfica
- **features/accidents/, disasters/, rescues/**: tabs para outros tipos de ocorrências
- **services/**: camada de acesso à API (api.ts, auth.ts, fireLocation.ts, rainLocation.ts, predictions.ts, dashboard.ts, import.ts)
- **components/**: componentes reutilizáveis (Navbar, FilterBar, Modal, TimeFilter, LoadingSpinner)

4.6 Tecnologias PAW

Tecnologia	Versão	Camada
Go	1.25	Backend — linguagem principal
Gin	v1.12	Backend — framework HTTP
GORM	v1.31	Backend — ORM para PostgreSQL
golang-jwt/jwt	v5.3	Backend — geração/validação JWT
excelize	v2.10	Backend — parsing de ficheiros XLSX
gin-contrib/cors	v1.7	Backend — middleware CORS
PostgreSQL	16	Base de dados relacional
PostGIS	latest	Extensão geográfica do PostgreSQL
React	19	Frontend — biblioteca de UI

Tecnologia	Versão	Camada
TypeScript	6	Frontend — tipagem estática
Vite	8	Frontend — build tool
Leaflet / react-leaflet	1.9 / 5.0	Frontend — mapas interativos
Biome	2.4	Frontend — linter e formatter
Bun	latest	Frontend — package manager e runtime

4.7 Rubricas PAW — Autoavaliação

Critério	Peso	Implementação
Arquitetura da aplicação	20%	Separação clara: cmd/internal/handlers/services/models, frontend por features
Implementação do backend (Go)	40%	Gin + GORM, JWT auth, CORS, importação async XLSX, endpoints REST completos
Implementação do frontend	20%	React 19 + TS, Leaflet, hooks customizados, AuthContext, rotas protegidas
Usabilidade e UX	10%	FilterBar, TimeFilter, LoadingSpinner, modais, CSS por componente
Documentação	10%	README completo com stack, API reference, env vars, docker instructions

5. PSR — Projeto em Sistemas e Redes

5.1 Containerização com Docker

Todos os serviços da plataforma estão containerizados com Dockerfiles dedicados:

- **backend/Dockerfile**: build multi-stage Go (builder **golang:latest** + runtime **alpine:latest**), compilação estática com `CGO_ENABLED=0`.
- **frontend/Dockerfile**: build multi-stage (builder **oven/bun** compila a SPA Vite + runtime **oven/bun** com **serve** para estáticos).
- **predictionEngine/Dockerfile**: build multi-stage Python 3.12-slim (builder instala dependências com **uv**, runtime copia apenas `.venv` + código da aplicação).
- Imagens externas: **postgres/postgis:latest**, **minio/minio:latest**.

5.2 Orquestração com Docker Compose

Dois ficheiros de composição cobrem os ambientes:

- **docker-compose.yaml** (produção): integração com Traefik, TLS via Cloudflare, rede Docker externa **proxy**.
- **docker-compose.dev.yaml** (desenvolvimento): portas expostas diretamente, sem TLS, rede interna **terrawatch-dev**.

Serviços orquestrados: **postgres_db**, **minio**, **go_backend**, **prediction_engine**, **react_frontend**. Dependências explícitas entre serviços (**depends_on**) garantem ordem de inicialização correta.

5.3 Reverse Proxy e TLS — Traefik

O Traefik atua como reverse proxy e gestor de certificados TLS, partilhado a nível do homelab:

- **Configuração estática** (`config/traefik/traefik.yaml`): endpoints **web** (HTTP, redireciona para HTTPS) e **websecure** (HTTPS), DNS challenge Cloudflare com `certResolver=cloudflare`, ACME storage em volume persistente, Docker provider com `exposedByDefault: false`.
- **Configuração dinâmica** via labels Docker nos serviços do TerraWatch: roteamento por hostname, TLS automático, balanceamento por porta interna.
- **Hostnames roteados**: `terrawatch.oshico.org` (frontend), `terrawatch-api.oshico.org` (backend), `terrawatch-pe.oshico.org` (prediction engine), `terrawatch-s3.oshico.org` (MinIO API), `minio.oshico.org` (MinIO console).
- Rede Docker externa **proxy** partilhada com outros serviços do homelab.

5.4 Pipeline CI/CD — GitHub Actions

O ficheiro `.github/workflows/deploy.yml` implementa um pipeline completo com dois jobs:

Job	Stage	Ferramentas	Descrição
checks	Install + Lint Frontend	Bun + Biome	Instala dependências frontend, executa linter Biome
checks	Build Frontend	Bun + Vite	Compila a SPA React em produção
checks	Vet Backend	Go 1.25	Análise estática do código Go (go vet)

Job	Stage	Ferramentas	Descrição
checks	Build Backend	Go 1.25	Compila o binário Go (verifica ausência de erros, -o /dev/null)
checks	Check ML	uv (astral-sh)	Sync de dependências Python com cache do venv
deploy	SSH Deploy	appleboy/ssh-action	SSH para homelab: git pull + docker compose up -d --build

O deployment ocorre automaticamente em pushes para **main** (não em PRs), via SSH para o servidor homelab usando credenciais armazenadas nos GitHub Secrets (HOST, USERNAME, KEY, PORT). O script de deploy faz **git pull** e re-build das imagens Docker.

5.5 Triggers da Pipeline

- Push para branches **main** ou **development**
- Pull Request para branches **main** ou **development** (apenas checks, sem deploy)
- Trigger manual via **workflow_dispatch**

5.6 Deployment em Produção

A plataforma está deployada num homelab self-hosted, acessível em <https://terrawatch.oshico.org>.

Infraestrutura partilhada do homelab

O homelab executa os seguintes serviços de infraestrutura, definidos em `~/Projects/homelab/docker-compose.yaml`:

Serviço	Função
Traefik	Reverse proxy central, TLS automático (Cloudflare DNS challenge), dashboard protegido por basic auth
Cloudflare DDNS	Atualização dinâmica de DNS (subdomínio wildcard *.oshico.org)
Portainer	Gestão visual de containers Docker
Prometheus	Recolha e armazenamento de métricas (retenção 15 dias, remote-write habilitado)
Loki	Agregação centralizada de logs (retenção 7 dias, TSDB)
Alloy	Coletor de telemetria: logs Docker via loki.source.docker, métricas do sistema via prometheus.exporter.unix
Grafana	Visualização de métricas e logs, datasources Prometheus e Loki auto-provisionados

Redes Docker

Duas redes Docker externas:

- **proxy** — rede partilhada entre Traefik e todos os serviços expostos (incluindo TerraWatch). O TerraWatch liga-se a esta rede via docker-compose.yaml, herdando o roteamento Traefik automaticamente.
- **monitoring** — rede interna para comunicação entre Prometheus, Loki, Alloy e Grafana.

Volumes persistentes

- **postgres_data** — dados da base de dados PostgreSQL/PostGIS
- **minio_data** — object storage para artefactos ML
- **grafana_data** — dashboards e configuração Grafana
- **prometheus_data** — séries temporais de métricas
- **loki_data** — logs agregados
- **traefik_acme** — certificados TLS (ACME)

Ambiente de produção

O ambiente de produção usa Docker Compose com:

- Traefik para HTTPS automático e reverse proxy
- Volumes persistentes para base de dados e object storage
- Rede Docker externa **proxy** partilhada com serviços do homelab
- Health checks nos containers e **restart: unless-stopped**
- Logs centralizados via Docker (consumidos por Alloy → Loki → Grafana)

5.7 Variáveis de Ambiente e Configuração

Toda a configuração sensível é gerida via variáveis de ambiente (**.env**), nunca hardcoded no código. O ficheiro **.env.example** documenta todas as variáveis necessárias:

- Credenciais da base de dados (POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB)
- Segredos JWT (JWT_SECRET, JWT_ACCESS_EXPIRY, JWT_REFRESH_EXPIRY)
- Credenciais MinIO (MINIO_ROOT_USER, MINIO_ROOT_PASSWORD)
- Configuração do backend (DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, DB_PORT)
- Domínio de produção (DOMAIN) e URLs das APIs (VITE_API_URL, VITE_PE_API_URL)
- Bucket ML (PE_BUCKET_NAME)

5.8 Tecnologias PSR

Tecnologia	Utilização
Docker	Containerização de todos os serviços
Docker Compose	Orquestração multi-container (dev e produção)
Traefik	Reverse proxy central, roteamento HTTPS, TLS automático via Cloudflare DNS challenge
GitHub Actions	Pipeline CI/CD automatizado
appleboy/ssh-action	Deploy via SSH para homelab
astral-sh/setup-uv	Gestão de ambiente Python na CI
MinIO	Object storage S3-compatible para modelos ML
Cloudflare DDNS	Atualização dinâmica de DNS wildcard
Portainer	Gestão visual de containers
Prometheus	Recolha e armazenamento de métricas do sistema e containers
Loki	Agregação e consulta centralizada de logs

Tecnologia	Utilização
Grafana	Dashboards de monitorização (métricas + logs), datasources auto-provisionados
Alloy	Coletor de telemetria (logs Docker + node exporter)
Homelab (self-hosted)	Ambiente de produção em VPS
Cloudflare DNS	Gestão de DNS e certificados TLS (ACME DNS-01 challenge)

5.9 Rubricas PSR — Autoavaliação

Critério	Peso	Implementação
Containerização com Docker	25%	Dockerfiles multi-stage dedicados para backend (Go + Alpine), frontend (Bun multi-stage) e ML service (Python + uv multi-stage); imagens externas para PostGIS e MinIO
Pipeline CI/CD	25%	GitHub Actions com lint (Biome), build (Vite + Go), vet e sync Python; deploy automático via SSH para homelab em pushes à main
Deployment funcional	20%	Plataforma em produção em https://terrawatch.oshico.org com TLS automático, volumes persistentes, object storage S3
Reprodutibilidade do sistema	20%	docker-compose.yaml + docker-compose.dev.yaml + .env.example documentado; qualquer pessoa pode replicar o ambiente com docker compose up
Monitorização e logs	10%	Stack completa de observabilidade: Prometheus (métricas), Loki (logs centralizados), Grafana (dashboards auto-provisionados), Alloy (coletor Docker + node exporter); health checks; restart: unless-stopped

6. Resumo de Todas as Tecnologias Utilizadas

Tecnologia	Versão	Disciplina	Papel no Projeto
Go	1.25	PAW	Backend API REST
Gin	v1.12	PAW	Framework HTTP para Go
GORM	v1.31	PAW	ORM / acesso à base de dados
golang-jwt/jwt	v5.3	PAW	Autenticação JWT
excelize	v2.10	PAW	Parsing de ficheiros XLSX
gin-contrib/cors	v1.7	PAW	CORS middleware
bcrypt (x/crypto)	v0.50	PAW	Hash seguro de passwords
PostgreSQL	16	PAW/PSR	Base de dados relacional
PostGIS	latest	PAW	Extensão geográfica de PostgreSQL
React	19	PAW	Biblioteca de UI (frontend)
TypeScript	6	PAW	Tipagem estática no frontend
Vite	8	PAW	Build tool do frontend
Leaflet / react-leaflet	1.9 / 5.0	PAW	Mapas interativos
Biome	2.4	PAW	Linter e formatter (frontend)
Bun	latest	PAW/PSR	Package manager e runtime JS
Python	3.12+	IART	Linguagem do motor ML
FastAPI	>=0.115	IART	Framework REST para ML service
Prophet	>=1.1.6	IART	Modelo de previsão de séries temporais
Pandas	>=2.2	IART	Preprocessamento de dados
NumPy	>=2.0	IART	Operações numéricas
SQLAlchemy	>=2.0	IART	Acesso à BD no motor ML
MinIO SDK (Python)	>=7.2	IART/PSR	Armazenamento de modelos ML
Uvicorn	>=0.32	IART	Servidor ASGI
Docker	latest	PSR	Containerização de serviços
Docker Compose	latest	PSR	Orquestração multi-container
Traefik	latest	PSR	Reverse proxy + TLS automático
GitHub Actions	latest	PSR	Pipeline CI/CD
MinIO (server)	latest	PSR	Object storage S3-compatible
Cloudflare DNS	—	PSR	DNS + certificados TLS (ACME DNS-01 challenge)
Cloudflare DDNS	—	PSR	Atualização dinâmica de DNS wildcard

Tecnologia	Versão	Disciplina	Papel no Projeto
uv (astral-sh)	latest	PSR	Gestão de venv Python na CI
Portainer	latest	PSR	Gestão visual de containers Docker
Prometheus	latest	PSR	Recolha e armazenamento de métricas
Loki	latest	PSR	Agregação centralizada de logs
Grafana	latest	PSR	Dashboards de monitorização (métricas + logs)
Alloy	latest	PSR	Coletor de telemetria (logs Docker + node exporter)